IP Reputation API

Version	Date	Author	Description
1.0	5-28-2014	Andy Walker	Initial Docs
2.0	6-6-2014	Andy Walker	Adding expiration and renewal to /blacklist and adding /whitelist routes updated parameter names and descriptions
3.0	6/3/2016	James Su	Reputation is on PostgreSQL now - no MongoDB needed. API is also updated.

1. Index

- 1. Index
- 2. General Conventions
  - 2.1 Requests
  - 2.2 Responses
- 3. Blacklist Routes
  - 3.1 /blacklist/get
    - 3.1.1 Request
    - 3.1.2 Response
  - 3.2 /blacklist/add
    - 3.2.1 Request
    - 3.2.2 Response
  - 3.3 /blacklist/classifications
    - 3.3.1 Request
    - 3.3.2 Response
  - 3.4 /blacklist/exclude
    - 3.4.1 Request
    - 3.4.2 Response
  - 3.5 /blacklist/renew
    - 3.5.1 Request
    - 3.5.2 Response
  - 3.6 /blacklist/expire
    - 3.6.1 Request
    - 3.6.2 Response
- 4. Whitelist Routes
  - 4.1 /whitelist/get
    - 4.1.1 Request
    - 4.1.2 Response
  - 4.2 /whitelist/add
    - 4.2.1 Request
    - 4.2.2 Response
  - 4.3 /whitelist/delete
    - 4.3.1 Request
    - 4.3.2 Response
- 5. Status Codes

2. General Conventions

2.1 Requests

All requests are listed as tables, describing how data is to be sent to the server including the method, as well as the parameters and their types. Parameters which are underlined are required, and those that can (optionally) appear multiple times are followed by **[...]**

Parameters or sections of data will appear in **`fixed-width-bold`** throughout the text.

Method	**URL**
`POST`	`prefix/this_route`

e.g.:

Param	**Info**
required	`Required parameter`
`can_multiple [...]`	`Can appear multi-times`
req_can_multiple `[...]`	`It's required and can appear multi-times`

2.2 Responses

All normal API responses will be returned as JSON data. Messages from the server, as in the case of errors, will be in the **`MSG`** field:

Status	**Response**
`200`	`{ "MSG":"messages", "THIS":["JSON","data"] }`
`400`	`{"MSG":"You screwed up, buddy"}`
`404`	`{"MSG":"Not found, buddy"}`

3. Blacklist Routes

3.1 /blacklist/get

Retrieve data for entry/entries from the blacklist database

3.1.1 Request

Method	URL
POST	blacklist/get

Param	Type
entry [...]	string

entry:

Can be one or many hosts (ip addresses, domains or fully-qualified URLs) to search for.

3.1.2 Response

All entries located will be indexed by the string used to search for them in the post. If no entries are found, a **404** will be returned. **NOT_FOUND** will contain an array of requested items that were not located when searching the blacklist database.

Status	Sample Response

200	{
	"NOT_FOUND" : [
	"1.1.1.1",
	"unknownhost.com"
	],
	"badbadsite.com" : {
	"public" : "false",
	"classifications" : [
	"malware",
	"bogon"
	],
	"class_id" : 1,
	"expiration" : "2016-05-31 15:07:02.321397-04",

	"hostname" : "badbadsite.com",
	"rev" : 18,
	"stale" : "false",
	"last_seen" : "2016-05-02 14:25:44.271465-04",
	"primary_source" : "VRT",
	"metadata" : {
	"VRT" : {
	"comment" : "some comment again",
	"added_by" : "JamesSu"
	}
	},
	"excluded" : "false",
	"seen_by" : {
	"VRT" : 1
	},
	"first_seen" : "2016-05-02 14:25:44.271465-04",
	"disposition" : 3,
	"manual_classifications" : [
	"malware",
	"bogon"
	]
	}
	}

404	{"MSG":"No matching entries found"}

3.2 /blacklist/add

Add an entry to the database or update its classification(s) and expiration.

3.2.1 Request

Method	URL
POST	blacklist/add

Param	Type
entry [...]	string
classification [...]	comma-separated string
comment	string
author	string

`entry`

An IP address (dotted quad), domain or URL to add to the database. Entries will be given basic sanity checking to ensure they conform to proper dotted-quad/domain name/URI format.
If multi entries provided, each entry will be created or updated with the same classification(s), comment and author.

`classification(s)`

One or more classifications to assign to the entry. These **must** be valid classifications as retrieved from the **`/blacklist/classifications`** route below.

`comment`

The reason this host is being added or modified

`author`

The user responsible for this addition *(**MUST be lowercase, no spaces.** Will disappear when integrated with sandbox)*

3.2.2 Response

If the entry has not been seen before, we return code **`201`** - Created. If it has, we return **`202`** - Accepted. In either case, we return the new or updated ISO 8601 expiration date. If the entry is invalid in some way, we return code **`400`** - Bad Request.

Status	Response

201	{
	"MSG":"Entry created",
	"expiration" : "2013-05-08T10:05:02"
	}

202	{
	"MSG":"Entry updated",
	"expiration" : "2013-05-08T10:05:02"
	}

400	{"MSG":"Invalid IP address: 256.0.0.1" } {"MSG":"Blacklist only takes hostname, IP address or URL."}

3.3 /blacklist/classifications

Returns a list of valid classifications allowed by the blacklist.
NOTE: No parameters permitted.

3.3.1 Request

Method	URL
GET	blacklist/classifications

3.3.2 Response

The response will always be code **200** - OK, and will contain an array of valid classifications

Status	Response
200	["attackers", "bogon", "bots", "cnc", "dga", "exploitkit", "malware", "open_proxy", "open_relay", "phishing", "response", "spam", "suspicious", "tor_exit_node"]

3.4 /blacklist/exclude

Immediately exclude an entry from the blacklist. The entry may continue to be updated by automated systems or VRT members, but it will no longer appear in the blacklist unless explicitly reinstated with **`/blacklist/renew`**.

3.4.1 Request

Method	URL
`POST`	`blacklist/exclude`

Param	Type
entry	`string`

`entry`
An IP address (dotted quad), domain or URL to be excluded from the blacklist.

3.4.2 Response

If the entry does not exist, we return the standard **`404`** - Not Found. If the entry has already been excluded, we return code **`400`** - Bad Request.

Status	Response
`404`	`{"MSG":"No matching entries found"}`
`400`	`{"MSG":"Entry is already excluded"}`

3.5 /blacklist/renew

Immediately renew a blacklist entry. The entry may again appear in the list if updated by an automated process or if it has not passed its expiration, but will not have its expiration updated by this command. To do so, a subsequent call to **`/blacklist/add`** is required.

3.5.1 Request

Method	URL
`POST`	`blacklist/renew`

Param	Type
entry	`string`

`entry`
: An IP address (dotted quad), domain or URL to renew

3.5.2 Response

If the entry does not exist, we return the standard **`404`** - Not Found. If the entry has already been renewed or has never been expired, we return code **`400`** - Bad Request.

Status	Response
`404`	`{"MSG":"No matching entries found"}`
`400`	`{"MSG":"Entry is not excluded"}`

3.6 /blacklist/expire

Immediately set a blacklist entry expired.

3.6.1 Request

Method	URL
POST	blacklist/expire

Param	Type
Entry [...]	comma-separated string

entry

An IP address (dotted quad), domain or URL to renew

3.6.2 Response

If the entry does not exist, we return the standard **404** - Not Found.

Status	Response
404	{"MSG":"No matching entries found"}
200	{"MSG":"Entry has been set to expired."}

4. Whitelist Routes

4.1 /whitelist/get

Retrieve data for entry/entries from the whitelist database

4.1.1 Request

Method	URL
POST	whitelist/get

Param	Type
entry [...]	string

entry

Can be one or many IP addresses

4.1.2 Response

All entries located will be indexed by the IP used to search for them in the post. If no entries are found, a **404** will be returned. **NOT_FOUND** will contain an array of requested items that were not located when searching the whitelist database.

Status	Response
200	`{ "NOT_FOUND" : [ "goodsite.com" ], "75.125.228.68" : { "source" : "awalker", "comment" : "keegy.com", "range" : "75.125.0.0/16", "ident" : "alexa001" } }`
404	`{"MSG":"No matching entries found"}`

4.2 /whitelist/add

Add an entry to the whitelist database

4.2.1 Request

Method	URL
`POST`	`whitelist/add`

Param	Type
entry	`string`
comment	`string`
author	`string`
`identifier`	`string`

`entry`

An IP address (dotted quad) or domain to add to the database. Entries will be given basic sanity checking to ensure they conform to proper dotted-quad/domain name format.

`comment`

The reason this host is being added

`author`

The user responsible for this addition *(**MUST be lowercase, no spaces.** Will disappear when integrated with sandbox)*

`identifier`

A short string to allow for blocks of entries to be associated with each other. Especially useful for large VRT-maintained blocks, automated processes, or groups that periodically change. For example, if there is a large block automated import that we periodically wish to refresh, assigning an `identifier` to these means we can delete them all at once and add them again.

4.2.2 Response

If the entry does not exist in the whitelist, we return **`201`** - Created. If the entry already exists, we return **`409`** - Conflict. If the entry is invalid in some way, we return code **`400`** - Bad Request.

Status	Response
201	`{ "MSG":"Entry created" }`
409	`{ "MSG":"Entry exists", "entry" : { "source" : "awalker", "comment" : "keegy.com", "range" : "75.125.0.0/16", "ident" : "alexa001" } }`
400	`{"MSG":"Invalid IP or CIDR address: 256.0.0.1/900" }`

4.3 /whitelist/delete

Remove an entry from the whitelist database

4.3.1 Request

Method	URL
POST	whitelist/delete

Param	Type
entry	string
comment	string

`entry`

An IP address (dotted quad) or domain to be removed from the database. Entries will be given basic sanity checking to ensure they conform to proper dotted-quad/domain name format.

`comment`

The reason this host is being removed

4.3.2 Response

If successfully deleted the entry from the whitelist, we return **`202`**. If the entry does not exists, we return **`409`**. If the entry is invalid in some way, we return code **`400`** - Bad Request.

Status	Response

202	`{`
	`  "MSG":"Entry deleted"`
	`}`

409	`{`
	`  "MSG":"Entry not exists in whitelist",`
	`}`

400	`{"MSG":"Invalid IP or CIDR address: 256.0.0.1/900" }`

5. Status Codes

All status codes are standard HTTP status codes. The below ones are used in this API.

2XX - Success of some kind
4XX - Error occurred in client's part
5XX - Error occurred in server's part

Status Code	Description
200	OK
201	Created
202	Accepted (Request accepted, and queued for execution)
400	Bad request
401	Authentication failure
403	Forbidden
404	Resource not found
405	Method Not Allowed
409	Conflict
412	Precondition Failed
413	Request Entity Too Large
500	Internal Server Error
501	Not Implemented
503	Service Unavailable